

# Task 2: Frontend Performance and Problem Solving

Ajibola Akelebe · Frontend Developer, Branda V2 · October 2026

When a Next.js site slows down, I don't start by guessing. I measure first: Lighthouse on a throttled mobile profile, real-user numbers from Vercel Speed Insights, and the Network and Performance tabs in Chrome. Then I fix the biggest cause and measure again.

In my experience the problems in the brief usually come from a few habits. Here is what I would look for and what I would do, with examples from how I built the Branda V2 assessment.

## Likely causes

| Problem                 | What usually causes it                                                                                            |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| Slow first load         | Too much JavaScript, pages rendered on every request when they could be static, render-blocking fonts and scripts |
| Slow images             | Full-size images sent to phones, no <code>sizes</code> , the hero image discovered late                           |
| Poor mobile performance | Heavy JavaScript on mid-range phones, big images over mobile data, third-party tags                               |
| Too many API requests   | A request on every keystroke, the same data fetched by several components, nothing cached                         |
| Unnecessary re-renders  | State kept too high in the tree, components subscribed to a whole store instead of the piece they need            |
| Large bundle            | <code>"use client"</code> placed too high, heavy libraries for small jobs, analytics and chat widgets             |

## What I would do

### Images

- Use `next/image` everywhere. It serves AVIF or WebP at the right size for each screen.
- Give every image an honest `sizes` value so a phone never downloads a desktop image. In Branda, product cards say they are half the screen on mobile and a quarter on desktop.
- Preload only the hero image ( `preload` in Next 16, which replaces `priority` ). Everything else lazy loads by default.
- Put every image in a box with a fixed aspect ratio, so the page doesn't jump when it loads.

### Lazy loading, dynamic imports and code splitting

- The App Router already splits code per route. The job is not to undo that.
- Load heavy, rarely used parts on demand with `next/dynamic` : a 3D preview, a rich-text editor, a chart on an admin page.

- Keep `"use client"` at the edges. In Branda the product card is a server component, and only its small `"+"` button runs in the browser.

## Caching

- Cache catalogue data on the server with `use cache` and a tag, so updating one price clears that entry without a rebuild.
- Let the CDN serve anything that is the same for every visitor. Branda's market home pages are fully static.
- Give hashed static files a long cache life. Never cache the cart or checkout.
- In the browser, use TanStack Query with sensible stale times so going back to a page doesn't refetch everything.

## API requests

- **Deduplicate.** On the server, React's `cache()` means a page and its metadata asking for the same service hit the database once. TanStack Query does the same on the client.
- **Batch.** One request for "the four related services", not four requests.
- **Debounce, or don't fire at all.** Search shouldn't send a request per keystroke. In Branda, search runs when you press Enter, and each filter change updates the URL once.

## Re-renders

- Profile with React DevTools before changing anything. Blind memoisation adds code without fixing the cause.
- Keep state close to where it's used, and subscribe to the smallest slice of a store. Branda's cart button reads only the item count, so a cart change re-renders that one button, not the whole header.
- Memoise only what the profiler shows is expensive.

## Bundle size

- Run the bundle analyser and start with the biggest surprise.
- Swap libraries for platform features where you can. Branda's cart drawer, menu and search use the native `<dialog>` and popover, and all the animation is CSS, so no animation library ships.
- Load analytics, ads and chat with `next/script` after the page is interactive. At my previous workplace, where I set up consent-aware GTM and GA4, tags wait their turn instead of competing with the first paint.

## Rendering strategy

| Strategy                     | When I use it                        | In Branda                                                           |
|------------------------------|--------------------------------------|---------------------------------------------------------------------|
| Static (SSG)                 | Pages that are the same for everyone | Market home pages and all service pages, built ahead of time        |
| ISR / tagged revalidation    | Content that changes now and then    | Catalogue and prices, once they come from a CMS or API              |
| Server rendering per request | Pages that depend on the request     | The catalogue with filters in the URL, order confirmation           |
| Streaming                    | Pages with one slow section          | Wrap the slow part in <code>Suspense</code> so the rest shows first |

| Strategy         | When I use it              | In Branda              |
|------------------|----------------------------|------------------------|
| Client rendering | Private, interactive state | Cart and checkout form |

## Mobile performance

I build for a mid-range Android phone on mobile data, because that is how many people in Nigeria will use Branda. I've done this before: IndexedDB caching tuned for 3G on Heka IQ, and offline browsing for shoppers on patchy connections in the Zora Market app. In practice that means less JavaScript, smaller images, and testing on a throttled profile and a real phone, not just a fast laptop.

## Core Web Vitals

- **LCP (main content shows fast):** hero image preloaded and correctly sized, fonts through `next/font`, static HTML from the CDN.
- **INP (taps respond fast):** less JavaScript on the main thread, no heavy work inside click handlers, third-party scripts deferred.
- **CLS (nothing jumps):** space reserved for images, loading skeletons that match the final layout, no banners pushed in above content.

After each release I check real-user numbers, not just Lighthouse. A good lab score doesn't help if real users on 3G are still waiting.